

AURA

All-in-one Unified Routine Assistant

Product Requirements Document — Personal Life Management Dashboard (MERN Stack)

Author	Owner	Status	Version
100 Days of Code — Capstone Project	Solo Developer	Draft	1.0

1. Overview

AURA (All-in-one Unified Routine Assistant) is a full-stack MERN (MongoDB, Express, React, Node.js) web application that consolidates everyday personal management — tasks, habits, expenses, notes, and workouts — into a single dashboard. Instead of juggling four or five separate apps, users get one place to plan their day, track their progress, and see a consolidated view of their life at a glance.

This project is being built as a capstone within a 100 Days of Code challenge, following a foundation in CRUD operations, MongoDB, and authentication. It is scoped to reinforce those fundamentals while introducing new concepts: multi-resource data modeling, MongoDB aggregation pipelines, nested subdocuments, and dashboard-style data visualization.

2. Problem Statement

People commonly use separate apps for to-dos, habit tracking, budgeting, journaling, and workout logging. This fragmentation makes it hard to get a single view of one's day or week, and increases the friction of staying consistent. AURA addresses this by unifying these functions under one authenticated account and one dashboard view.

3. Goals & Non-Goals

Goals

- Allow a user to manage tasks, habits, expenses, notes, and workouts from one account.
- Provide a dashboard that summarizes activity across all modules at a glance.
- Practice and demonstrate multi-model schema design, authentication, and MongoDB aggregation.
- Ship a deployed, portfolio-ready full-stack application.

Non-Goals (out of scope for v1)

- Social features (sharing, following other users, public profiles).
- Native mobile apps (web-responsive only for v1).
- Third-party integrations (e.g., bank sync, calendar sync, wearables) — may be considered post-v1.

- Team/multi-user collaboration on the same data.

4. Target User

Individuals who want a simple, self-hosted-feeling way to stay organized — students, early-career professionals, and self-improvement-minded users who currently spread this activity across multiple apps (e.g., Todoist, a budgeting app, a fitness app, and a notes app) and want it consolidated.

5. Feature Requirements

5.1 Authentication

Secure account creation and session management scoped per user.

- Users can sign up with name, email, and password (hashed with bcrypt).
- Users can log in and receive a JWT used to authorize subsequent requests.
- All module data (tasks, habits, expenses, notes, workouts) is scoped to the logged-in user via req.user.id.
- Protected routes redirect unauthenticated users to the login page.

5.2 Tasks

A standard to-do list with prioritization and due dates.

- Create, read, update, delete tasks.
- Each task has: title, description, due date, priority (low/medium/high), status (pending/complete), category.
- Task list can be filtered by status, priority, and category, and sorted by due date.
- Dashboard shows tasks due today and overdue tasks.

5.3 Habit Tracker

Daily check-ins with streak tracking to encourage consistency.

- Create, read, update, delete habits (e.g., 'Read 20 minutes', 'Drink water').
- Users can check in a habit once per day.
- System calculates current streak and longest streak per habit.
- Dashboard shows active streaks and today's incomplete habits.

5.4 Expense Log

Lightweight personal expense tracking with category breakdowns.

- Create, read, update, delete expense entries: amount, category, date, note.
- View monthly total spend and spend broken down by category (via aggregation pipeline).
- Dashboard shows this month's total spend and top spending category.

5.5 Notes / Journal

Quick freeform notes for daily reflection or reference.

- Create, read, update, delete notes: title, body, date, optional tags.
- Full-text search across note titles and bodies.
- Dashboard shows the most recent note.

5.6 Workout Tracker

Gym session logging with nested exercise/set data and progress tracking.

- Create, read, update, delete workout sessions: date, name (e.g., 'Push Day'), list of exercises.

- Each exercise contains a name and a list of sets (reps, weight).
- Save reusable workout templates to speed up logging.
- View progress for a given exercise over time (e.g., max weight on Bench Press by date), rendered as a line chart.
- Aggregation endpoint for weekly training volume (sum of reps $\times$ weight) and workout frequency.
- Dashboard shows this week's workout count vs. a weekly goal, and the most recent session.

5.7 Dashboard

A single summary view pulling from all modules.

- Displays: tasks due today, active habit streaks, this month's spend, most recent note, and this week's workout summary.
- Includes at least two charts: expense breakdown by category, and a workout progress line chart.
- All dashboard data is served by dedicated aggregation endpoints rather than fetched and computed client-side.

6. Data Model (High Level)

All resources reference the User model via a `userId` field, except Workout sub-data which is embedded.

Model	Key Fields	Relationship
User	name, email, passwordHash	Parent of all other models
Task	title, description, dueDate, priority, status, category, userId	References User
Habit	name, checkIns[dates], currentStreak, longestStreak, userId	References User
Expense	amount, category, date, note, userId	References User
Note	title, body, tags[], date, userId	References User
Workout	date, name, exercises: [{ name, sets: [{ reps, weight }] }], userId	References User; exercises/sets embedded

7. Tech Stack

Layer	Technology
Frontend	React (Vite), React Router, Axios, Tailwind CSS, Recharts
Backend	Node.js, Express.js, JWT + bcrypt for auth
Database	MongoDB Atlas, Mongoose ODM
Deployment	Frontend: Vercel · Backend: Render or Railway · DB: MongoDB Atlas
Optional	node-cron for daily reminder emails/digests

8. Non-Functional Requirements

- Security: passwords hashed with bcrypt; JWT-protected routes; no user can access another user's data.
- Performance: dashboard should load its summary data via aggregation endpoints in under ~500ms on typical data volumes.
- Responsiveness: usable on both desktop and mobile browser widths.
- Reliability: API returns clear error messages and appropriate HTTP status codes for invalid input or auth failures.

9. Milestones / Build Roadmap

#	Milestone	Output
1	Plan schemas & wireframes	Schema sketches, page wireframes
2	Backend foundation	Express server, MongoDB connection, JWT auth

3	Core CRUD modules	Tasks, Habits, Expenses, Notes APIs
4	Workout module	Nested exercise/set schema + endpoints
5	Aggregation endpoints	Dashboard summary + chart data endpoints
6	React frontend	Auth pages + one page per module
7	Dashboard & charts	Unified dashboard with Recharts visualizations
8	Polish, test, deploy	Deployed app + repo + portfolio writeup

10. Success Metrics (Personal Project Context)

- All five modules (Tasks, Habits, Expenses, Notes, Workouts) fully functional with CRUD + auth.
- Dashboard correctly aggregates and displays live data from all modules.
- Application successfully deployed and accessible via a public URL.
- Codebase is clean enough to walk through in a technical interview, including schema design rationale.